Project 2 - Gramática definitiva

Hecho por: Juan Diego Montañez y Natalia Salazar Tena

Definición del lenguaje:

$$G = \langle N, \Sigma, S_0, P \rangle$$

N: Program, Declaration, ObjectDecl, TypeParams, Variance, ConstructorParams, Modifier, MethodAssign, Parameters, MainMethod, Block, Statement, VarDecl, Type, ObjectInstantiation, ObjectBody, OverrideDecl, Arguments, ControlFlow, Pattern, Expression, LogicExp, LogicExp', RelExp, RelExp', AddExp, AddExp', MultExp, MultExp', ExpExp, ExpExp', UnaryExp, ArrowExp, ArrowExp', BaseExp, BaseExp', Atom, Literal, IntLiteral, BooleanLiteral, StringLiteral, StringChar, Symbol, Interpolation, Identifier, ReservedWord, Letter, Digit.

Σ:

Palabras reservadas: "object", "extends", "val", "var", "method", "main", "new", "override", "if", "then", "else", "for", "and", "or", "neg", "true", "false", "in"

Operadores y Puntuación: "+", "-", "*", "/", "%", "**", "<", ">", "<=", ">=", "<>", "=", "<-", "->", "=>", "(", ")", "[", "]", "{", "}", ":", ".", ",", ";", "!", "?", "$", "${", ""'

S_0: Program

P = Reglas de produccion

Program	::= Declaration* MainMethod Declaration*
Declaration	::= ObjectDecl | MethodAssign
ObjectDecl	::= "object" Identifier (TypeParams)? ("extends" Identifier)? (ConstructorParams)?
TypeParams	::= "[" (Variance)? Identifier ("," (Variance)? Identifier)* "]"
Variance	::= "+" | "-"
ConstructorParams	::= "(" Modifier Identifier ":" Type ("," Modifier Identifier ":" Type)* ")"
Modifier	::= "val" | "var"
MethodAssign	::= Identifier "." Identifier "=" "method" "(" (Parameters)? ")" ":" Type (Block)?
Parameters	::= Identifier ":" Type ("," Identifier ":" Type)*
MainMethod	::= "method" "main" Block
Block	::= "{" (Statement)* "}"
Statement	::= VarDecl | Expression
VarDecl	::= Modifier Identifier (":" Type)? "=" (Expression | Block)
Type	::= Identifier ("[" Type ("," Type)* "]")?
ObjectInstantiation	::= "new" Type ("(" (Arguments)? ")")? (ObjectBody)?
ObjectBody	::= "{" (OverrideDecl)* "}"

OverrideDecl	::= "override" Identifier "(" (Parameters)? ")" ":" Type "=" (Block | Expression)
Arguments	::= Expression ("," Expression)*
ControlFlow	::= "if" Expression "then" Expression "else" Expression | "for" "(" Pattern ("<-" | "in") Expression ")" Expression
Pattern	::= Identifier | "(" Identifier ("," Identifier)* ")"
Expression	::= LogicExp
LogicExp	::= RelExp LogicExp'
LogicExp'	::= "and" RelExp LogicExp' | "or" RelExp LogicExp' |
RelExp	::= AddExp RelExp'
RelExp'	::= "<" AddExp RelExp' | ">" AddExp RelExp' | "<=" AddExp RelExp' | ">=" AddExp RelExp' | "<>" AddExp RelExp' | "=" AddExp RelExp' |
AddExp	::= MultExp AddExp'
AddExp'	::= "+" MultExp AddExp' | "-" MultExp AddExp' |
MultExp	::= ExpExp MultExp'
MultExp'	::= "*" ExpExp MultExp' | "/" ExpExp MultExp' | "%" ExpExp MultExp' |
ExpExp	::= UnaryExp ExpExp'
ExpExp'	::= "**" UnaryExp ExpExp' |
UnaryExp	::= "neg" UnaryExp | "-" UnaryExp | ArrowExp
ArrowExp	::= BaseExp ArrowExp'
ArrowExp'	::= "->" BaseExp ArrowExp' | "=>" BaseExp ArrowExp' |
BaseExp	::= ObjectInstantiation | ControlFlow | Atom BaseExp'
BaseExp'	::= "." Identifier ("(" (Arguments)? ")")? BaseExp' | "(" (Arguments)? ")" BaseExp' |
Atom	::= Literal | Identifier | "(" Expression ")"
Literal	::= IntLiteral | BooleanLiteral | StringLiteral
IntLiteral	::= Digit (Digit)*
BooleanLiteral	::= "true" | "false"
StringLiteral	::= '"' (StringChar)* '"'
StringChar	::= Letter | Digit | " " | Symbol | Interpolation

Symbol	::= ":" | "." | "," | ";" | "!" | "?" | "-" | "+" | "*" | "/" | "=" | "<" | ">" | "(" | ")" | "[" | "]" | "{" | "}"
Interpolation	::= "${" Expression "}" \| "$" Identifier
Identifier	::= Letter (Letter | Digit)* siempre que la cadena no sea una ReservedWord
ReservedWord	::= "object" | "extends" | "val" | "var" | "method" | "main" | "new" | "override" | "if" | "then" | "else" | "for" | "and" | "or" | "neg" | "true" | "false" | "in"
Letter	::= "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h" | "i" | "j" | "k" | "l" | "m" | "n" | "o" | "p" | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x" | "y" | "z" | "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H" | "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P" | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X" | "Y" | "Z"
Digit	::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"